Two-Point Parked Lock-In

Master Reference

Acquisition methods · timing breakdown · sensitivity
open defects · work log

Project NV compact magnetometer, RFSoC4x2 + QICK-DAWG
Repository `nv_magnetometer_project`, branch `main`
Sessions covered 2026-08-04, 2026-08-06, 2026-08-14
Written 2026-08-17

Supersedes
`docs/2026-08-06_twopoint_timing/TIMING_AND_NOISE_ANALYSIS.md` and `docs/2026-08-14_twopoint_methods/TWOPOINT_METHODS_AND_TIMING.md`, both of which remain in the repository as the record of what was believed when. Chapter 12 lists every disagreement.

Regenerate every number `python scripts/analyze_twopoint_0814.py`

The three findings, in one paragraph each

Rate. An `acquire()` call costs 10–11.4 ms of host round-trip whatever the program does, and the FPGA work runs underneath it. Halving the readout window cut the FPGA work by 4× and changed the rate by 0.4%. The averaged mode therefore sits at ∼88 Hz at any setting, and averaging is *free* up to the call floor.
Sensitivity. The reference-photodiode fix bought **4.1**×: the burst-mode noise floor went from 158 to $\mathbf{38\,nT/\sqrt{Hz}}$. Burst is the best operating point available today.
Bandwidth. Streaming delivers **1041.7 Hz** with a gapless, duplicate-free rep axis — the 1 kHz target is met. Its noise floor is ∼6× the burst floor for reasons not yet identified, which is the single highest-value open item.

Chalach Kasemtantikul

Contents

Chapter 1

How to read this document

1.1 What this is

This is the single reference for the two-point parked-frequency lock-in on the NV compact magnetometer: how each acquisition method works, exactly where the acquisition time goes, what sensitivity each method delivers, which defects are open, and what has been changed in the code so far.

It merges and supersedes two earlier documents:

- `docs/2026-08-06_twopoint_timing/TIMING_AND_NOISE_ANALYSIS.md` — the 2026-08-06 session: timing budget, readout-window optimum, discovery of the burst-mode replay.
- `docs/2026-08-14_twopoint_methods/TWOPOINT_METHODS_AND_TIMING.md` — the 2026-08-14 session: the three modes compared, the per-call floor, two open defects.

Both remain in the repository as the record of what was believed when. Where they disagree with this document, this document wins, and Chapter 12 lists every such disagreement explicitly.

1.2 Where the numbers come from

Every measured value here is regenerated by one script from committed CSV files:

```
python scripts/analyze_twopoint_0814.py
```

It writes `docs/2026-08-14_twopoint_methods/tables/*.csv` and `figures/*.png`; this document quotes those tables. Nothing is hand-typed, so re-running the script after a new session refreshes the numbers rather than invalidating them. Values that can only be measured on the rig are marked **OPEN** and named as such — there are five of them, all in Chapter 11.

1.3 The three findings that matter most

1. The acquisition rate is set by the host, not by the readout window

An `acquire()` call costs **10–11.4 ms of host round-trip whatever the program does**, and the FPGA pulse work runs *underneath* that rather than adding to it. Halving the readout window from 213 µs to 120 µs cut the FPGA work from 9.80 ms to 2.40 ms and changed the rate by nothing (87.5 → 86.8 Hz).
Consequences: shortening τ is not a rate lever in averaged mode; averaging more reps is *free* up to the call floor; and beating ∼90 Hz requires amortising the call across many samples, which is what burst and streaming do. See Chapter 7.

2. The photodiode fix bought 4.1×, and burst mode shows it

After the reference-photodiode gain was corrected and the signal diode stopped saturating, the burst-mode noise floor went from **158 to 38 $\mathrm{nT}/\sqrt{\mathrm{Hz}}$** — a factor of 4.1 at the same two parked points. Burst is the best operating point available today by a wide margin. See Chapter 8.

3. Streaming reaches 1 kHz, but its noise floor is ∼6× burst

Streaming delivered **1041.7 Hz** with a gapless, duplicate-free rep axis across 125 000 reps — the 1 kHz target is met. Its noise floor, however, sits a flat ∼6× above burst across 10–500 Hz for reasons not yet identified. Chapter 9 lists what has been ruled out and the one-minute rig test that decides it.

1.4 Status at a glance

Item	State	Number	Where
Averaged mode rate	**DONE**	86.8–88.8 Hz	§4
Burst mode rate	**DONE**	4167 Hz in-burst	§5
Streaming rate	**DONE**	1041.7 Hz	§6
Best measured noise floor	**DONE**	$38\,\mathrm{nT}/\sqrt{\mathrm{Hz}}$ (burst)	§8.2
Readout window chosen	**DONE**	120 µs	§8.1
Timing model	**DONE**	max(host, FPGA)	§7.2
Free averaging implemented	**DONE**	42 reps fit, 10 used	§7.3
Per-mode post-processing	**DONE**	3 pipelines	§9.5
Notebook reorganised	**DONE**	4A/4B/4C + 5A/5B/5C	§10.4
Burst replay root cause	**OPEN**	49.9% of batches	§9.1
Streaming noise excess	**OPEN**	∼6× burst	§9.2
Per-acquire noise term	**OPEN**	does not average down	§8.4
ABBA ordering benefit	**OPEN**	never tested on hardware	§2.2
Calibrated system sensitivity	**OPEN**	no field reference yet	§2.4

Chapter 2

The measurement, from photons to Δf

Before any timing discussion, this is what a "sample" actually is. All three acquisition methods produce the same quantity by the same arithmetic; they differ only in how many FPGA repetitions go into one sample and how the host collects them.

2.1 Two parked frequencies on one flank pair

A single ODMR resonance is fitted (Lorentzian, from the Step 1 reference sweep), giving its centre f_0, half-width γ, baseline B, and the two maximum-slope points f_- and f_+ on either flank. Rather than sweeping, the microwave source is *parked* alternately at f_- and f_+ and the photoluminescence is read at each.

$$z_\pm = \frac{\text{counts at } f_\pm}{\text{normalisation}} \qquad \text{normalised PL at each parked point} \tag{2.1}$$

$$L = z_+ - z_- \qquad \text{the lock-in signal} \tag{2.2}$$

$$\Delta f = \frac{L - (b_+ - b_-)}{m_- - m_+} \qquad \text{shift of the resonance, in MHz} \tag{2.3}$$

where $b_\pm$ are the calibration baselines at the parked points and $m_\pm$ the flank slopes, both measured off the reference sweep. Field is then $\Delta B = \Delta f / \gamma_{\text{NV}}$ with $\gamma_{\text{NV}} = 28.024\,\text{kHz}\,\mu\text{T}^{-1}$ along the tracked NV axis.

2.1.1 Why the difference and not one flank

If the resonance shifts by δ, z_- moves down the negative-slope flank and z_+ moves up the positive-slope one: the shift appears with opposite sign in the two channels and *adds* in L. A change in laser power or collection efficiency scales both channels the same way and *cancels* in L to first order.

This works, and by a large factor. In the 2026-08-06 heat-gun/motor run, z_- and z_+ swung **13.4% peak-to-peak together** while the difference stayed flat — a common-mode rejection ratio of **32×**.

2.1.2 The noise consequence

Because Δf is linear in L,

$$\sigma(\Delta f) = \frac{\sigma(z_+ - z_-)}{|m_- - m_+|}. \tag{2.4}$$

Two things follow that recur throughout this document:

1. **Collapsed dip depth costs sensitivity directly.** The slopes come from the calibration. If the resonance moves off the parked pair, or the laser/MW level drifts, the real slopes flatten while the assumed ones do not, and $\sigma(\Delta f)$ grows in proportion. Every acquisition cell now prints the measured dip depth at both parked points against the calibrated value.
2. **Post-processing must act on the difference, not on the channels.** Filtering z_- and z_+ independently breaks the cancellation. This is not hypothetical: see §9.5.1.

2.2 The residual first-order term, and ABBA

The cancellation is not perfect, because the two points are not read at the same instant. In the default `"forward"` ordering the rep visits f_- then f_+, one readout window apart, *always in the same direction.* A common-mode drift d per unit time therefore contributes $+d\,\tau$ to L every rep: the estimator is differentiating the common mode.

`"abba"` ordering emits f_-, f_+, f_+, f_- and folds slot 0 with slot 3 and slot 1 with slot 2. A linear drift contributes $+d, +2d, +3d, +4d$ across the four slots, and the folding cancels it exactly. It doubles the readouts per rep, so one `"abba"` rep costs and averages exactly two `"forward"` reps — the cancellation is paid for in rep count, not in time.

OPEN — ABBA has never been tested on hardware

Implemented and verified in simulation; free at equal averaging. Given the 32× rejection already measured and the 13.4% common-mode swing, the residual first-order term is plausibly significant, but no A/B has been run. Pass criterion and protocol: §11.1, check S7.

2.3 The two sensitivity conventions, fixed once

Two numbers are quoted throughout, and they are not the same thing.

$\eta = \sigma_B\sqrt{2\,\Delta t}$ — the “implied” sensitivity
: The white-noise amplitude spectral density that a measured standard deviation σ_B at sample spacing Δt would correspond to. Cheap to compute from any trace. *Only meaningful if the trace is actually white*: drift inflates σ without raising the noise density.

Measured floor — the median ASD in the flat band
: Read directly off a Welch periodogram between 10 Hz and $0.4 f_s$. Makes no whiteness assumption.

Both are reported side by side, and **when they disagree the floor is the one to trust**; the ratio is a direct readout of how drift-dominated the trace is. For example the 2026-08-14 burst run gives $\eta = 56$ but a floor of $38\,\mathrm{nT}/\sqrt{\mathrm{Hz}}$, a ratio of 1.5 — the trace is not white.

The convention is fixed in code, in `notebook_modules/twopoint_spectra.py`, whose PSD normalisation is verified against Parseval's theorem (the integral of the PSD reproduces the variance to 0.02%) and against a synthetic tone of known amplitude.

Correction — an earlier $\sqrt{2}$ and an earlier $\sqrt{\mathrm{reps}}$

The 2026-08-06 document used $\eta = \sigma_B\sqrt{\Delta t}$, smaller than the convention above by $\sqrt{2}$. Its figures are restated here in the $\sqrt{2\Delta t}$ convention where they are compared with 2026-08-14 numbers.

Separately, the first draft of that document paired a *reps-averaged* sweep-point σ with a *single-rep* time, understating η by $\sqrt{\mathrm{reps}} \approx 10$. That was corrected on 2026-08-12 by cross-calibrating against the burst runs; see §8.1.

2.4 What is not being claimed

Every sensitivity figure in this document is a **readout-chain** figure: it comes from the measured scatter of Δf and the ODMR-derived slopes. None of them is a calibrated system sensitivity, because that requires a known applied field, which has not been done. The distinction matters for the project's sensitivity claims and is the reason `NV Project/wiki/concepts/sensitivity.md` keeps these numbers separate from the roadmap target.

Chapter 3

What all three methods share

The three acquisition methods run the *same* FPGA program and read the *same* accumulated buffer. They differ only in how many repetitions the host takes per call and how it drains the buffer. Everything in this chapter is therefore common to all three, and the per-method chapters that follow only have to describe the differences.

3.1 The FPGA program: one rep, step by step

`MultipointLockinODMR.body()` (`notebook_modules/multipoint_lockin_program.py`, lines 191–236) emits the following assembly *per parked frequency*, i.e. per *emission slot*:

#	ASM step	What it does	Time cost
1	`mw_frequency_register.set_to(f)`	Retunes the MW generator at runtime. This is the trick that lets 2 (or 16) non-uniform frequencies live in one program instead of one upload each.	a few tProc cycles (∼ns)
2	`pulse(ch=mw_channel, t=0)`	Constant-amplitude MW pulse, length set equal to the readout window.	**0** — runs concurrently with step 3
3	`trigger_no_off(adcs, pins=[laser_gate], width=readout_integration_treg, t=0)`	Opens the laser gate *and* arms the ADC accumulator for `readout_integration_treg` clock cycles. **This step is the rep.**	$\boldsymbol{\tau}$ (120 µs at the current default)
4	`trigger(pins=[laser_gate], width=relax_delay_treg, t=...treg)`	Closing trigger: drives the laser gate low at the end of this slot.	3.3 µs nominal, absorbed in practice
5	`wait_all()`	Waits for the ADC to finish writing.	∼0
6	`sync_all(relax_delay_treg)`	Advances the tProc time cursor.	3.3 µs nominal, absorbed in practice

3.1.1 How many slots there are

`multipoint_pair_order`	`multipoint_skip_reference`	Slots/rep	Readouts/rep
`"forward"`	`True` *(current default)*	2	**2**
`"forward"`	`False`	2	4
`"abba"`	`True`	4	**4**
`"abba"`	`False`	4	8

`multipoint_skip_reference = True` drops the per-slot “MW-off” reference readout. It is on because the MW cannot actually be gated off on this setup, so the reference was a contaminated copy of the signal rather than a clean normaliser — it halved the FPGA time and removed a systematic. With it on, z is normalised against a stored constant (`ref_norm_counts` from the calibration JSON) instead of a per-shot reference.

3.1.2 The one timing formula everything else derives from

$$\boxed{t_{\text{rep}} \;=\; n_{\text{slots}} \times n_{\text{readouts per slot}} \times \tau} \tag{3.1}$$

At the current default (2 slots, 1 readout each, $\tau = 120\,\mu\text{s}$):

$$t_{\text{rep}} = 2 \times 1 \times 120\,\mu\text{s} = 240\,\mu\text{s} \qquad \Longrightarrow \qquad 4167\,\text{Hz per rep.}$$

The relax windows do not appear in Eq. 3.1, and that is measured

An earlier model added the two relax windows per slot, giving $n_{\text{slots}} \times (\tau + 2 \times 3.3\,\mu\text{s})$. The measurement does not support it. Across the burst runs the cadence is $426.0\,\mu\text{s}$ at $\tau = 213\,\mu\text{s}$ — exactly 2×213 — and $240.0\,\mu\text{s}$ at $\tau = 120\,\mu\text{s}$ — exactly 2×120. The relax and sync windows overlap the readout rather than following it. `time_per_rep()` was corrected accordingly; `include_relax=True` restores the old sum as a conservative upper bound.

3.2 What the accumulated buffer holds

The ADC does not hand back samples; it hands back a *sum*. For each readout trigger, the firmware accumulates `readout_integration_treg` raw ADC samples into one 64-bit integer entry, and writes one such entry per readout per rep. So after a run of `reps` repetitions with R readouts per rep, the buffer holds reps $\times$ R integers, laid out as `(reps, nsweep, R)` — with `nsweep = 1` here.

`analyze_results()` then does two divisions:

$$\text{stored value} \;=\; \frac{\text{integer accumulator sum}}{\underbrace{\texttt{readout\_integration\_treg}}_{\text{always}} \times \underbrace{\texttt{reps}}_{\text{averaged mode only}}} \tag{3.2}$$

Two consequences, both used heavily later:

1. **The stored counts are τ-independent in the mean** (they are a mean ADC level, not a total), which is why the counts do not grow when τ is raised. Their *variance* does fall as $1/\tau$, which is the sensitivity benefit of a longer window.
2. **The exact FPGA readout time is recoverable from the recorded numbers.** This is the measurement that settled the timing question; it gets its own section.

3.3 Recovering the exact readout time from the data

Because the numerator of Eq. 3.2 is an integer, every stored count is an integer multiple of

$$\frac{1}{D}, \qquad D \equiv \texttt{treg} \times \texttt{reps}.$$

Searching for the smallest divisor D that makes *all* of a run's counts integral therefore recovers D exactly, with no reliance on what the notebook claims it configured. The readout time follows:

$$\boxed{t_{\text{ADC per call}} \;=\; n_{\text{slots}} \times \frac{D}{f_{\text{clk}}}, \qquad f_{\text{clk}} = 307.2\,\text{MHz}} \tag{3.3}$$

The clock is pinned independently by two runs: $213\,\mu\text{s} \leftrightarrow \texttt{treg} = 65434$ and $120\,\mu\text{s} \leftrightarrow \texttt{treg} = 36864$, both giving $307.2\,\text{MHz}$.

Only the product is identifiable. $D = 36864$ is (`treg` $= 36864$) $\times$ (`reps` $= 1$) and (`treg` $= 768$) $\times$ (`reps` $= 48$) equally well. That is sufficient, because Eq. 3.3 depends only on D. In *burst* mode there is no rep averaging, so D *is* `treg` and the readout window follows directly — which is how the 2026-08-14 burst run was confirmed to be at $\tau = 120.0\,\mu\mathrm{s}$, exactly as configured.

Implemented as `twopoint_postprocess.recover_readout_quantum()` and `readout_seconds()`.

Why this matters practically

It replaced a 12% error. The burst cadence had been estimated as `acq_seconds / n_samples` over the real batches, which reads 269 µs against a true 240 µs because it includes the host share of each acquire. That 12% stretch went straight into the time axis, the quoted rate, and the frequency scale of every spectrum.

3.4 The host RPC chain

`NVAveragerProgram.acquire()` (`qickdawg/nvpulsing/nvaverageprogram.py`, lines 163–304) issues this sequence of Pyro remote calls to the board on **every** call:

Stage	Call	Notes
1	`config_all(soc, load_envelopes, reset, load_mem)`	Uploads the program. `load_mem=True`, so the binary and data memory are re-sent on every call. Until 2026-08-12 this raised `TypeError` on every call (wrong keyword) and fell through to defaults.
1a	→ `soc.start_src("internal")`	inside `config_all`
1b	→ `soc.stop_tproc(lazy=not reset)`	**documented no-op on tProc v1** unless `reset=True`
1c	→ `load_envelopes`, `load_bin_program`, `load_mem`	re-uploaded each call
2	`soc.start_src(start_src)`	again, from `acquire`
3	`config_bufs(soc, enable_avg=True, enable_buf=False)`	arms the accumulator
4	`soc.reload_mem()`	no-op unless tProc v2
5	`soc.start_readout(total_count, ...)`	starts the board-side worker *and* the tProc
6	`soc.poll_data()` $\times N$	**blocks here while the FPGA runs**
7	Pyro deserialise → `analyze_results()` reshape	host-side

The critical structural fact: stage 6 is where the FPGA time goes

The host does not wait for the FPGA *after* finishing its own work — it waits *inside* stage 6, which is part of the chain. So the FPGA time is **concurrent with** the host chain, not additive to it, up to the point where the FPGA run outlasts the rest of the chain.
That single fact is the whole content of Chapter 7, and it is what the 2026-08-06 analysis got wrong.

OPEN — the per-stage split has never been measured

The stage list above is read off the source. The *time* each stage takes can only be measured on the rig, by wrapping the soc proxy so every remote call is timed. `scripts/profile_twopoint_acquire.py` does exactly that and has never been run, because the RFSoC is not reachable

from the analysis machine. Until it is, the chain is known only in total (10–11.4 ms) and not stage by stage.
This is the single largest gap in this document. Run:

```
python scripts/profile_twopoint_acquire.py -ip 192.168.0.103
```

3.5 Python outside the call

Measured directly as (loop period − `acq_seconds`), per batch:

Run	Python outside `acquire()`	Share of period
2026-08-14 τ=120, 10 reps	0.067 ms	0.58%
2026-08-14 τ=120, 23 reps	0.056 ms	0.49%
2026-08-06 τ=213, 23 reps	0.058 ms	0.51%

Row-dict construction, the conversion arithmetic and the history append together cost well under a tenth of a millisecond. **The Python side is not a bottleneck in any mode** and the earlier vectorisation of the per-sample conversion did its job. It is listed here so that it can be ruled out rather than suspected.

The exception is the CSV write, which is not per-sample and is covered per mode.

Chapter 4

Method A — Averaged mode

Notebook cell	Step 4A (analysis: Step 5A)
Code path	`prog.acquire(progress=False)`
One call returns	the *mean* over `reps` reps: one sample
Measured rate	**86.8–88.8 Hz**, at every τ and rep count tested
Rate limited by	the host call (∼11.4 ms), *not* the readout window
Duty cycle	20.9% at 10 reps, 48.3% at 23 reps, 85.7% at τ=213/23 reps
Measured floor	274–501 nT/$\sqrt{\text{Hz}}$ (varies 1.8× between runs)
Open defects	none

4.1 Mechanics

The program runs `reps` repetitions. `analyze_results()` averages over the rep axis (Eq. 3.2 divides by `reps`) and returns one number per parked frequency. The host then converts that pair into one Δf and one CSV row, and immediately calls `acquire()` again.

The cost of this design. The rep axis is *discarded*. Those `reps` repetitions were already independent time samples spaced by t_{rep}; averaging them away throws that time resolution out. Methods B and C exist to keep it.

4.2 Timing budget, component by component

Three operating points, all measured. Read the **Adds?** column first: it is what distinguishes this mode from the other two.

Operating point 1: $\tau = 120\,\mu\text{s}$, 10 reps — what actually ran on 2026-08-14

Component	Time	Share	Adds?	How obtained
ADC integration (10 × 240 µs)	2.400 ms	20.9%	no	exact, Eq. 3.3, D=368640
of which MW pulse	0 ms	0%	no	concurrent with the readout
of which relax/sync	∼0 ms	∼0%	no	absorbed; measured, §3.1
`acquire()` call, fastest observed	10.302 ms	89.8%	—	p05 of `acq_seconds`
`acquire()` call, median	**11.408** ms	**99.4%**	**yes**	median of `acq_seconds`
`acquire()` jitter (p95−p05)	1.624 ms	14.2%	yes	host/network scheduling
Python outside `acquire()`	0.067 ms	0.58%	yes	period − `acq_seconds`
CSV flush stall (worst)	215.8 ms	—	yes	2 events > 3× median period
TOTAL per sample	**11.475** ms	100%		→ **87.1 Hz**

Read this table as a maximum, not a sum

2.400 + 11.408 = 13.8 ms, but the measured period is 11.475 ms. The two do not add, because the ADC integration happens *inside* stage 6 of the host chain (§3.4). The period is

$$\begin{aligned}\text{period} &= \max(\text{host chain},\ \text{reps} \times t_{\text{rep}}) \ + \ \text{Python outside} \\ &= \max(11.408,\ 2.400) + 0.067 \ = \ 11.475\,\text{ms}.\end{aligned}$$

Only 2.400 ms **of every 11.475** ms **is spent collecting photons.** The remaining 79% is the host talking to the board.

Operating point 2: $\tau = 120\,\mu s$, 23 reps

Component	Time	Share	Note
ADC integration: 23 × 240 µs	5.520 ms	48.3%	D=847872, exact
`acquire()` call, fastest	10.266 ms	89.7%	
`acquire()` call, median	11.384 ms	99.5%	
`acquire()` jitter	1.819 ms	15.9%	
Python outside	0.056 ms	0.49%	
CSV flush stalls	0 ms	—	none in this run
TOTAL per sample	**11.440** ms	100%	→ **87.4 Hz**

Operating point 3: $\tau = 213\,\mu s$, 23 reps — the 2026-08-06 default

Component	Time	Share	Note
ADC integration: 23 × 426 µs	9.798 ms	85.7%	D=1504982, exact
`acquire()` call, fastest	10.218 ms	89.4%	
`acquire()` call, median	11.369 ms	99.5%	
`acquire()` jitter	1.877 ms	16.4%	
Python outside	0.058 ms	0.51%	
CSV flush stall (worst)	219.5 ms	—	4 events
TOTAL per sample	**11.428** ms	100%	→ **87.5 Hz**

4.2.1 The three side by side — the decisive comparison

	τ=120, 10 reps	τ=120, 23 reps	τ=213, 23 reps	spread
ADC integration	2.400 ms	5.520 ms	9.798 ms	**4.1**×
Period	11.475 ms	11.440 ms	11.428 ms	**1.004**×
Rate	87.1 Hz	87.4 Hz	87.5 Hz	1.005×
Duty cycle	20.9%	48.3%	85.7%	4.1×

This one row is the whole finding

The ADC integration time varies by **4.1**× across these three operating points. The period varies by **0.4%**. There is no readout-window or rep-count setting that makes this mode faster than ∼88 Hz.

4.3 The CSV flush, and why it was fixed

The worst single stall in these runs is 219.5 ms — nineteen missed samples. The cause was the old save routine calling `pd.DataFrame(rows).to_csv(...)` every 5 seconds, rewriting *every row collected so far*: $O(N^2)$ over a run. Of the 94 outlier batches across the 2026-08-06 session, **46 sat within 150** ms **of a 5-second boundary**.

Replaced by `twopoint_runner.IncrementalCsvWriter`, which appends only the new rows (header once). On a 40 000-row run this is 10.2× cheaper and the stall is flat with run length instead of growing.

4.4 What to set

Parameter	Recommended	Why
`AVG_REPS_PER_BATCH`	`None`	Fills the measured call floor automatically via `reps_that_fit()`. At τ=120 that is **42 reps**, against the 10 actually used — see §7.3.
`AVG_CALIBRATE_FLOOR`	`True`	Measures the floor on this rig at startup; it already differs by 1.2 ms between the two sessions. Costs ∼0.3 s.
`readout_integration_tus`	120	Inside the flat sensitivity band (§8.1). Not a rate lever here.
`AVG_PAIR_ORDER`	`"forward"`	`"abba"` is free at equal averaging but untested (§2.2).
`AVG_SKIP_REFERENCE`	`True`	The MW-off reference is contaminated on this setup.
`AVG_SHOW_PLOT`	`False`	A live 3-panel redraw costs 20–100 ms, which at an 11 ms period would dominate the loop.

4.5 Failure modes and what warns you

Failure	Symptom	Guard
Resonance drifts off the parked pair	Dip depth collapses, $\sigma(\Delta f)$ grows as 1/depth (Eq. 2.4)	`report_calibration_validity()` prints measured vs calibrated depth and depth/noise; warns below 50% or ratio < 3
60 Hz mains	Appears at 26.8–28.8 Hz, indistinguishable from signal	`alias_report()` in Step 5A; **cannot be removed** (§8.5)
Averaging does not help	σ gets *worse* with more reps	**OPEN** — see §8.4
Peaking transient	Per-batch transient from an MW polarisation pulse at each batch boundary	`pre_init=False` on the live program, enforced by `assert`; primed once with a throwaway `pre_init=True` acquire
Stale buffer	Would show as a bit-identical repeat	Freshness guard; **zero occurrences** in 20 365 averaged batches

4.6 When to use it

When ∼88 Hz is enough. It is the only one of the three methods with no open defect, its timestamps are honest, and it has never been observed to replay a buffer. For drone sensing and MCG, where bandwidth is the point, it is the wrong choice — go to Method B or C.

Chapter 5

Method B — Burst mode

Notebook cell	Step 4B (analysis: Step 5B)
Code path	`prog.acquire(per_rep=True, reset_tproc=True)`
One call returns	*every* rep as its own sample: `reps` samples
Measured rate	**4167 Hz in-burst**, **3089 Hz net**
Rate limited by	the FPGA cadence; the host cost is amortised over the burst
Duty cycle	**74.1%**
Measured floor	**38 nT/$\sqrt{\text{Hz}}$** — best of the three
Open defects	**OPEN 49.9% of bursts are replays** (§9.1)

5.1 Mechanics

The FPGA program is identical to Method A's. The only change is that `analyze_results(per_rep=True)` *keeps* the rep axis: the second division in Eq. 3.2 is skipped, and the call returns a (reps × n_{freqs}) array. Every row is an independent time sample spaced by t_{rep}.

The ∼11 ms host chain is therefore paid once per *burst* of 500 samples instead of once per sample — a 500× amortisation, which is where the rate comes from.

The structural difference from Method A. Here the FPGA time and the host time *do* add. Stage 6 of the chain (§3.4) cannot return until the tProc has finished all 500 reps, which takes 120 ms — far longer than the rest of the chain. The mode is genuinely FPGA-bound, which is exactly what Method A is not.

5.2 Timing budget, component by component

Operating point: $\tau = 120\,\mu\text{s}$, 500 reps per burst (2026-08-14)

Component	Time per burst	Share	How obtained
Productive time			
ADC integration: 500 × 240 µs	**120.000** ms	81.7%	exact, Eq. 3.3
Overhead, per real burst			
Host RPC chain (adds here)	14.697 ms	10.0%	measured wall − exact FPGA
`acquire()` total, real burst	134.697 ms	91.7%	median of `acq_seconds`, real batches
p05 / p95	118.0 / 136.3 ms		spread is host jitter
Pure waste — the open defect			
Stale replay acquire	12.533 ms	8.5%	×185 batches
total run time wasted	**2.32** s of 30.11 s	7.7%	produces no usable data
Result			
Total per usable sample	**0.324** ms		30.11 s / 92 814 samples
Net rate	**3089 Hz**		
In-burst rate	**4167 Hz**		$1/t_{\text{rep}}$, exact
Duty cycle	**74.1%**		22.32 s of FPGA in 30.11 s

Where the 26% of non-productive time goes

Where the 30.11 s run went	Time	Share
ADC integration (productive)	22.32 s	**74.1%**
Host chain, 186 real bursts	2.73 s	9.1%
Stale replay acquires (waste)	2.32 s	7.7%
Flushes, row-0 drops, residual	2.74 s	9.1%

Fixing the replay defect would recover the 7.7% directly and would also remove the wasted host chain on those calls — roughly a 15% rate gain, taking the net rate from 3089 Hz toward ∼3600 Hz.

Operating point: $\tau = 213\,\mu s$, 500 reps per burst (2026-08-06)

Component	Time per burst	Share	Note
ADC integration: $500 \times 426\,\mu s$	213.001 ms	100.3%	exact
Host RPC chain	<2.13 ms	<1%	below the resolution of timing a 213 ms call
`acquire()` total, real burst	212.267 ms	100%	measured
Stale replay acquire	12.648 ms	6.0%	×106 batches = 1.34 s wasted
Total per usable sample	0.514 ms		
Net rate	1946 Hz		
In-burst rate	2347 Hz		
Duty cycle	82.9%		

Note that the host share is a *smaller* fraction here because the FPGA run is longer — the amortisation improves with burst length. That is the argument for large `BURST_REPS`, bounded only by how long a gap in the time series is acceptable and by the accumulated-buffer size.

5.2.1 Rate against burst length

With host chain $H \approx 14.7$ ms and stale overhead $S \approx 12.5$ ms per real burst, the net rate is

$$f_{\text{net}} = \frac{N}{N\,t_{\text{rep}} + H + S} \tag{5.1}$$

which at $\tau = 120\,\mu s$ gives:

`BURST_REPS`	FPGA per burst	Net rate	Duty cycle
100	24.0 ms	1961 Hz	47%
250	60.0 ms	2874 Hz	69%
500	**120.0** ms	**3413 Hz**	**82%**
1000	240.0 ms	3771 Hz	90%
2000	480.0 ms	3963 Hz	95%

(The 500-rep row predicts 3413 Hz against 3089 Hz measured; the difference is the CSV flushes and the dropped row-0 samples.) The returns flatten past ∼1000 reps, and the gaps grow: a 2000-rep burst leaves a 27 ms hole in the time series every 480 ms.

5.3 Three artefacts, and what each one is

The 2026-08-06 `Burst Mode.png` showed four visually distinct problems. All four are now explained, and three are fixed.

What you see	What it is	Status
Large spike, strictly every ∼0.45 s	Row 0 of each *stale* batch: **+1908 kHz ≈ +68 µT** mean. Real batches read −72 kHz at row 0, so this is the buffer read, not physics.	Fixed by dropping stale batches
Smaller, less regular spikes	**Genuine single-rep glitches, recorded twice**, because the replay reproduces them. E.g. −1443 kHz at sample 824 appears in both batch 11 and its replay batch 12.	De-duplicated; the glitches themselves are real and unexplained
Dense/sparse striping	The **timestamp model**. Samples were stamped across the *measured* acquire window: 13 ms for a stale batch, 425 ms for a real one — a **33**× **alternating compression** of the time axis, even though the true cadence is a constant 424.8 µs.	Fixed: timestamps now come from the cadence
Break every ∼5 s	The `SAVE_EVERY_SEC = 5.0` full-DataFrame rewrite. 267 ms in one run.	Fixed by the append-only writer
Row 0 runs 1.4% high in *real* batches too	A **genuine post-idle transient**: 1210.6 against 1193.4 ADC. Distinct from the replay, and present even in clean batches.	Dropped at acquisition and in post-processing

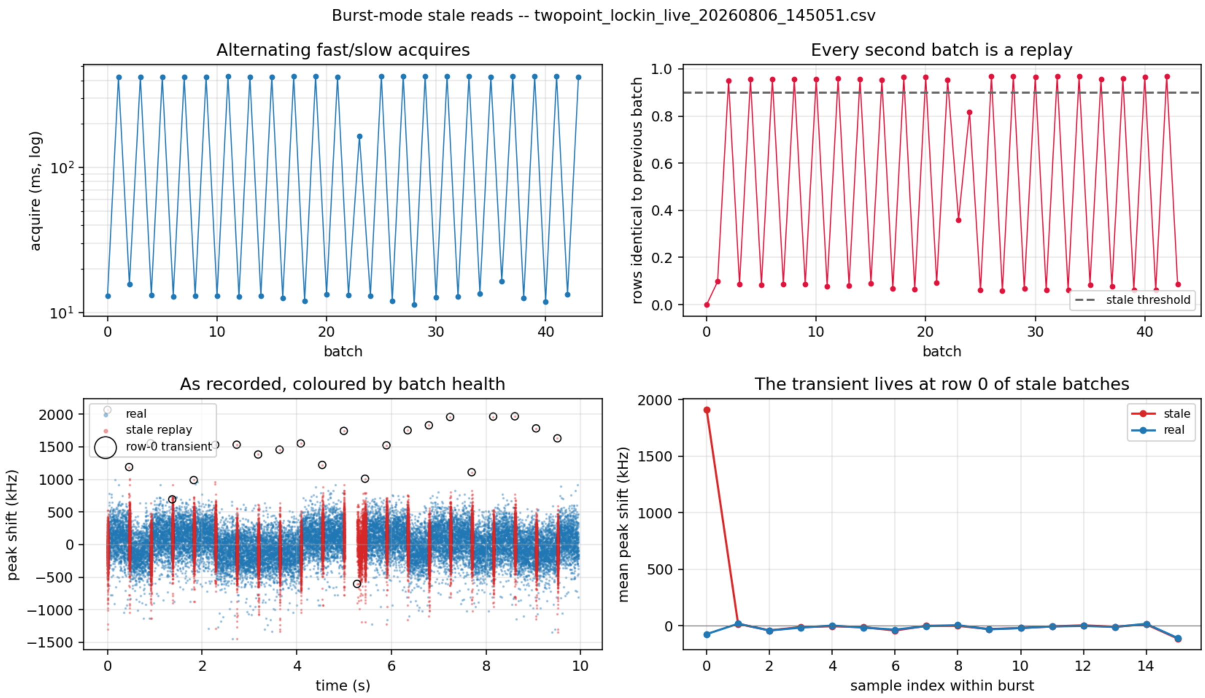

Figure 5.1: Burst-mode stale reads on 2026-08-06. The stale replays (red) compress into narrow vertical stripes because their 1000 samples are stamped across a 13 ms window while the real samples (blue) spread over 425 ms. Circled points are the row-0 transients — one every ∼0.45 s, exactly the periodicity seen in the original plot.

5.4 What to set

Parameter	Recommended	Why
`BURST_ON_STALE`	`"drop"`	Re-acquires, then *skips* the batch rather than writing a replay to the CSV. Costs duty cycle, not correctness.
`BURST_STALE_RETRIES`	2	The race is transient; a repeat call usually lands on a completed run.
`BURST_REPS`	500–1000	See the rate table above; returns flatten past 1000 and the time-series gaps grow.
`BURST_DROP_FIRST_REP`	`True`	Row 0 runs 1.4% high — a real transient.
`BURST_RESET_TPROC`	`True`	Kept available and harmless, but **measured not to fix the replay** (§9.1).

5.5 When to use it

Today, for best sensitivity: this is the mode. $38\,\mathrm{nT}/\sqrt{\mathrm{Hz}}$ at an effective 3.1 kHz beats streaming by ∼6× and averaged mode by 7–13×. The price is a 26% duty cycle loss and a ∼15 ms hole in the time series every 135 ms, which matters for a continuous MCG trace and does not matter for a noise-floor measurement.

If the replay defect is ever fixed, this mode gets ∼15% faster for free. If the streaming noise defect is fixed, Method C becomes strictly better for continuous recording.

Chapter 6

Method C — Streaming mode

Notebook cell	Step 4C (analysis: Step 5C)
Code path	`prog.stream(total_reps=N)` — a generator
One packet returns	38–500 reps, continuously, as they arrive
Measured rate	**1041.7 Hz** (or 4167 Hz with no host binning)
Rate limited by	the FPGA cadence alone
Duty cycle	**100.0%**
Measured floor	240 nT/$\sqrt{\text{Hz}}$
Open defects	**OPEN floor is ~6× burst** (§9.2); 25% PL droop (§6.5)

6.1 Mechanics

One `config_all`, one `start_readout(total_reps)`, then nothing but draining the streamer queue with `poll_data()` in a loop. The board-side worker thread transfers completed shots out of the accumulated buffer autonomously while the tProc keeps running.

Because nothing is re-armed mid-run:

- the replay race of Method B **cannot occur** — there is no second configuration to race against;
- there is **no inter-burst gap** — the FPGA never stops;
- timestamps follow the FPGA cadence **exactly**, rather than being reconstructed from host arrival times, which only record when the host happened to drain the queue.

`cfg.reps` is the run length. The tProc loops `reps` times and ends, so the rep count must be set before compilation and cannot change mid-run. `stream()` raises if `cfg.reps != total_reps` rather than silently truncating.

Host-side binning. `STREAM_TARGET_HZ` sets an integer number of reps to average on the host per output sample. Whole reps only, so the achieved rate is the cadence divided by an integer rather than exactly the target: at $\tau = 120\,\mu\text{s}$, a 1000 Hz target gives bin = 4 and therefore 1041.7 Hz. Leftover reps carry into the next packet, so no rep is dropped and no bin straddles a packet boundary.

6.2 Timing budget, component by component

Operating point: $\tau = 120\,\mu s$, bin = 4 (2026-08-14)

Component	Per output sample	Share	How obtained
ADC integration ($4 \times 240\,\mu s$)	**960.0 µs**	**100.0%**	exact, Eq. 3.3, D=147456
Host RPC chain	0 µs	0%	**paid once for the whole run**
Inter-packet gap	0 µs	0%	the FPGA does not stop between packets
Timestamp jitter	0.0 µs	0%	cadence-derived; min = max = 960.0 µs
TOTAL per output sample	**960.0 µs**	100%	measured spacing
Rate	**1041.67 Hz**		predicted 1041.67 Hz — *exact*
Duty cycle	**100.0%**		125 000 reps × 240 µs in 29.999 s

This is what a clean timing budget looks like

Every microsecond of the 960 µs sample period is ADC integration. The measured sample spacing has **min = max = 960.0 µs** — zero jitter, because the timestamps come from the cadence rather than from arrival times. Predicted and measured rate agree to the digit. Compare Method A, where 79% of the period is host overhead, and Method B, where 26% is overhead and waste.

6.2.1 Rate against binning

`bin`	Per output sample	Rate	Note
1	240 µs	**4166.7 Hz**	`STREAM_TARGET_HZ = None`; every rep kept
2	480 µs	2083.3 Hz	
4	**960 µs**	**1041.7 Hz**	what ran on 2026-08-14
8	1920 µs	520.8 Hz	
42	10.08 ms	99.2 Hz	equivalent averaging to Method A, at 100% duty

The last row is worth noting: streaming with `bin = 42` produces the same averaging as the recommended Method A operating point, at the same rate, but with 100% duty cycle instead of 21% — i.e. *if the streaming noise defect were resolved*, streaming would dominate Method A at every rate.

6.3 Verification: the guarantees, checked on the data

Streaming's value is its exactness, so Step 5C checks it rather than assuming it. On the 2026-08-14 run:

Check	Result	Meaning
`rep_index` gaps	**0**	no reps dropped; cadence timestamps valid throughout
`rep_index` step	4, uniformly	host binning consistent, no straddled bins
Duplicate rows	**0** of 31 250	the replay race really is absent
Reps collected	125 000 / 125 000	nothing lost
Packets	152	38–500 reps each, median 205
Duty cycle	100.0%	the FPGA never idled

`stream()` also now refuses to reshape a packet whose payload size does not match its own rep count. Such a packet would silently misalign every subsequent sample and mix the two parked frequencies together — which would look exactly like a flat broadband noise excess. No occurrence has been observed, but it would previously have been invisible.

6.4 The buffer-overflow constraint

The board worker transfers in strides of roughly 10% of the accumulated buffer and raises if unread samples reach `avg_maxlen`. `stream_headroom()` reports the actual margin at runtime rather than assuming it:

```
{avg_maxlen, reads_per_shot, shots_that_fit, seconds_that_fit, worker_stride_shots}
```

At $\tau = 120\,\mu\text{s}$ with one read per shot the cadence is ∼4 kHz, so even a modest buffer allows a poll interval of order a second. No overflow has been observed, and the 500-sample first packet is no noisier than the 205-sample ones, which argues against overflow being behind the noise excess.

6.5 The PL droop

Real, and specific to this mode

With no gap between acquires the sample heats. On the 2026-08-14 run the PL fell **25% over 30 s**, smoothly, from 993 to 744 ADC counts.
It is mostly common-mode and largely cancels in $z_+ - z_-$: it is 85% of the *common-mode* variance but only **2.8%** of the Δf variance. But it walks the operating point down the flank, so the slopes the conversion assumes drift out of validity over the run.
Step 5C detrends it (rolling median, corner $\sim f_s/\text{window}$) and reports the magnitude. The proper fix is a periodic re-zero during the run, which is not implemented.

6.6 When to use it

For anything that needs a *continuous* record above a few hundred Hz — MCG, drone transients — streaming is the only method that provides it: no gaps, exact timestamps, 100% duty cycle. Its rate is proven.

Its noise floor is currently ∼6× worse than burst, which for absolute sensitivity work makes burst the better choice today. **Resolving §9.2 is the single highest-value open item in this document**, because it would make streaming better than both other methods at every operating point.

Chapter 7

Timing, in full

Chapters 4–6 gave each method's budget. This chapter gives the evidence that the model behind them is right, and the practical consequences.

7.1 The evidence for a per-call floor

Applying the exact-recovery technique of §3.3 to all 21 usable averaged runs across three sessions (`tables/timing.csv`):

Session	Config	ADC integration	Fastest call	Median period	Rate
2026-08-04	D=654340	4.26 ms	9.35–9.53 ms	10.48–10.62 ms	94–95 Hz
2026-08-04	D=706560	4.60 ms	9.29 ms	10.84 ms	92 Hz
2026-08-04/06	D=1504982	9.80 ms	9.61–10.29 ms	10.93–11.54 ms	87–91 Hz
2026-08-14	D=368640	**2.40** ms	10.14–10.30 ms	11.26–11.52 ms	87–89 Hz
2026-08-14	D=847872	**5.52** ms	10.27 ms	11.39 ms	87.8 Hz

Two runs were excluded from the fit below because their loop period exceeds 1.5× their acquire time — most of it was spent outside `acquire()` (live plotting on, or operator interaction), so they say nothing about the call.

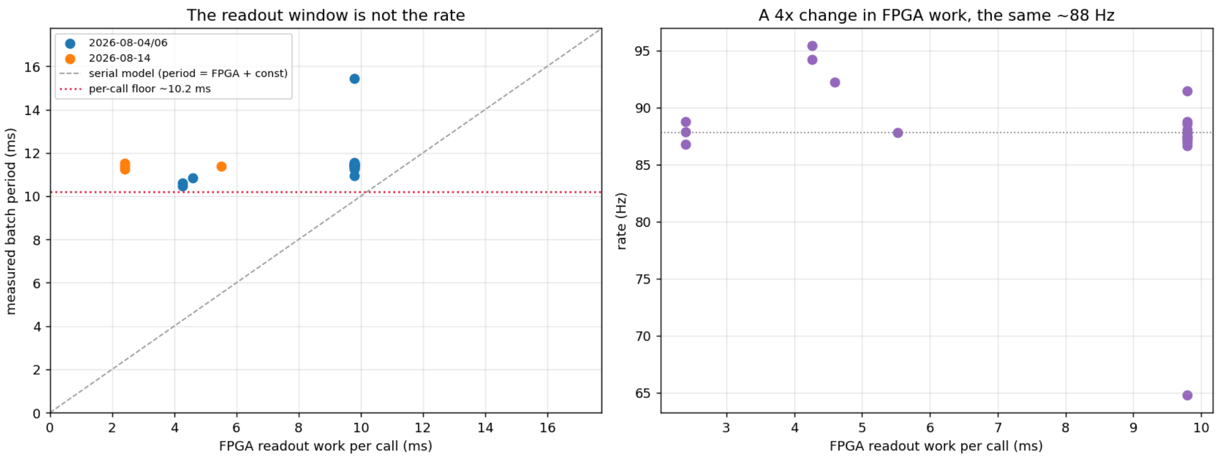

Figure 7.1: Left: measured batch period against FPGA readout work. The dashed line is the serial model the data does not follow; the red dotted line is the per-call floor. Right: the same data as rate — a 4× change in FPGA work leaves the rate at ∼88 Hz.

7.1.1 Argument 1 — the slope is not 1

Fitted *within* a session, because comparing across sessions would confound the FPGA change with the floor itself, which moved ∼1 ms between 2026-08-04/06 and 2026-08-14:

Session	FPGA work	Period	d(**period**)/d(**FPGA**)
2026-08-04/06	4.26 → 9.80 ms (2.3×)	10.55 → 11.43 ms	**+0.140**
2026-08-14	2.40 → 5.52 ms (2.3×)	11.38 → 11.39 ms	**+0.003**

A serial model period = host + FPGA requires slope = 1.000. Neither session comes close.

7.1.2 Argument 2 — no serial model fits at all

The 2026-08-14 runs used 2.40 ms of FPGA work, and their *fastest* call was 10.14 ms. Under a serial model that demands a host term of 7.74 ms, against the 1.60 ms the 2026-08-06 analysis reported for the same code on the same rig eight days earlier. There is no constant, non-negative host term that fits both sessions.

Under the max-model both are consistent: the host chain is ∼10–11.4 ms in both, and the FPGA work is simply hidden inside it.

7.1.3 Argument 3 — the wobble is entirely in the call

The instantaneous rate spans 81.4–100.1 Hz, which is what is seen live as "85–97 Hz". Decomposing it:

- The FPGA term is **deterministic** — 23 reps of identical pulse work vary by well under a microsecond, and the burst runs confirm the cadence is stable to 0.4% across runs.
- `acq_seconds` spans 10.1–12.1 ms (jitter 1.6–1.9 ms p95−p05).
- Python outside the call stays at 0.06 ms.

So **100% of the wobble is host/network scheduling inside `acquire()`** — Windows scheduling plus TCP/Pyro latency on stages 1–6.

7.2 The corrected model

$$\boxed{\text{period} \;=\; \max(\underbrace{H}_{\text{host call}},\ \underbrace{\text{reps} \times t_{\text{rep}}}_{\text{FPGA}}) \;+\; \underbrace{P}_{\text{Python, } \sim 0.06\,\text{ms}}} \tag{7.1}$$

with, on this rig,

$$H_{\text{typical}} = 11.4\,\text{ms}, \qquad H_{\text{floor}} = 10.1\,\text{ms}.$$

Two constants rather than one, because the two uses want different ends of the distribution: H_{typical} predicts the rate, H_{floor} sizes the free averaging (fill past the fastest call and even the quick calls become FPGA-bound).

7.2.1 Predicted against measured

τ	reps	FPGA	Old model	Eq. 7.1	Measured
213 µs	23	9.80 ms	160 Hz	**87.7 Hz**	87.5 Hz
120 µs	23	5.52 ms	400 Hz	**87.7 Hz**	87.8 Hz
120 µs	10	2.40 ms	180 Hz	**87.7 Hz**	86.8 Hz

The model giving the *same* answer for all three is the point.

`measure_host_floor()` calibrates H on the rig in about a second, using a `reps=1` probe whose FPGA work is 240 µs — so whatever the call still costs is the host chain, measured rather than inferred. Step 4A calls it at startup. Given that H already differs by 1.2 ms between the two sessions, the defaults above should be treated as a starting point.

7.3 Free averaging

The most actionable consequence in this document

If the FPGA work hides under the call, then **running fewer reps than fit is pure loss**. The call takes the same wall-clock time either way, so the unused milliseconds are photons that were never collected.

At $\tau = 120\,\mu\text{s}$, $t_{\text{rep}} = 240\,\mu\text{s}$, so $H_{\text{floor}}/t_{\text{rep}} = 10.1/0.240 =$ **42 reps** fit inside the floor. The 2026-08-14 runs used **10**.

reps	FPGA	Duty cycle	Rate	Note
10	2.40 ms	21%	87.7 Hz	what ran
23	5.52 ms	48%	87.7 Hz	
42	**10.08** ms	**88%**	**87.7 Hz**	the fill point
50	12.00 ms	100%	83.3 Hz	now FPGA-bound; rate starts to fall
100	24.00 ms	100%	41.7 Hz	

Implemented as `reps_that_fit()`, used automatically when `AVG_REPS_PER_BATCH = None`. `describe_timing()` prints the headroom:

```
2 freqs, order=forward (2 slots/rep), tau=120.0 us, 10 reps -> 240 us/rep, 2.40
ms FPGA work
averaged mode:  2.40 ms of FPGA work hides under the 10.1 ms per-call floor (24%
used) -> 87.7 Hz
FREE AVERAGING: 42 reps also fit inside the floor.  Raising 10 -> 42 costs no rate
and buys up to 2.0x in sigma if the noise is white.
```

The caveat is in that last clause

"*if the noise is white*" — and §8.4 shows it currently is not. The averaged mode's noise does not average down at all; it gets *worse* with more reps. Filling the budget is still free, so it is still worth doing, but do not expect the full $\sqrt{4.2} = 2.0\times$ until the per-acquire noise term is understood.

7.4 Knob by knob

Knob	Effect on rate	Effect on sensitivity	Verdict
`readout_integration_tus` (τ)	**None** in averaged mode below the floor; *is* the cadence in burst/stream	Flat for $\tau \leq 140\,\mu s$, worse above (§8.1)	Not a rate lever in Method A. Keep at 120 µs
`reps`	**None** below the floor; linear above	Should be $1/\sqrt{N}$; measured far less (§8.4)	Free averaging up to the floor
Acquisition method	**12–47**×	See Chapter 8	**The only real rate lever**
`multipoint_skip_reference`	2× (already on)	Removes a contaminated normaliser	Keep on
`multipoint_pair_order`	2× per rep for `"abba"`	Cancels linear common-mode drift; free at equal averaging	**OPEN**, untested
`n_freqs`	Linear	2 is the minimum for a two-point estimator	Fixed at 2
`BURST_REPS`	Sub-linear; flattens past ∼1000	None directly	500–1000
`STREAM_TARGET_HZ`	Sets the bin divisor exactly	$1/\sqrt{\text{bin}}$ if white	Free choice
`LIVE_SHOW_PLOT`	20–100 ms per redraw	None	Keep `False`
`SAVE_EVERY_SEC` rewrite	25–220 ms every 5 s	None	Fixed: append-only

7.5 The 1 kHz question, closed

The 2026-08-06 analysis concluded that 1 kHz was unreachable with one call per sample. That conclusion **survives the retraction and in fact gets stronger**: the per-call cost is ∼11 ms, not 1.6 ms, so a 1 ms period is unreachable by a factor of eleven rather than by 1.6.

The route it identified — one `start_readout` with continuous polling — was built, and on 2026-08-14 it delivered **1041.7 Hz** exactly as designed (§6.2). The rate target is met. Only its noise is open.

Chapter 8

Sensitivity and noise

8.1 The readout-window optimum

From 92 ODMR sweeps at 17 readout windows on 2026-08-06 (`docs/2026-08-06_twopoint_-timing/tables/integration_time.csv`). Lorentzian fits over 2840–2900 MHz; noise taken from the point-to-point scatter of the off-resonance wings, which is immune to slow drift across the sweep.

τ (µs)	contrast	σ_z	$\sigma_z\sqrt{\tau}$	$\sigma(\Delta f)$ kHz	t_{rep} µs	η nT/$\sqrt{\mathrm{Hz}}$
50	0.541	3.01e-3	0.0213	51.3	109	186 ± 14
90	0.517	2.13e-3	0.0202	37.0	189	$\mathbf{177 \pm 14}$
110	0.515	1.90e-3	0.0199	33.7	229	$\mathbf{178 \pm 8}$
120	0.514	1.98e-3	0.0217	34.9	249	192 ± 17
130	0.507	1.70e-3	0.0194	30.6	269	$\mathbf{175 \pm 15}$
140	0.506	1.71e-3	0.0202	30.6	289	181 ± 7
170	0.513	2.01e-3	0.0262	35.6	349	232 ± 5
190	0.495	2.09e-3	0.0288	38.4	389	264 ± 22
213	0.531	1.52e-3	0.0222	25.9	435	188 ± 14

Three results:

1. **Contrast is flat** (0.49–0.54) across the whole range. A longer window buys no signal.
2. **$\sigma_z\sqrt{\tau}$ is flat below ∼140 µs (0.0207) and rises above it (0.0252).** Below 140 µs the readout averages down as $\sqrt{\text{time}}$; above it, the extra window length collects drift instead of statistics. *This is the physical reason an optimum exists.*
3. **η is flat at 185 ± 7 nT/$\sqrt{\mathrm{Hz}}$ for $\tau \leq 140$ µs**, degrading to 232 ± 20 over 150–200 µs. Individual windows inside the flat band are not distinguishable — the sweep-to-sweep scatter at fixed τ is ±14, larger than the spread between them.

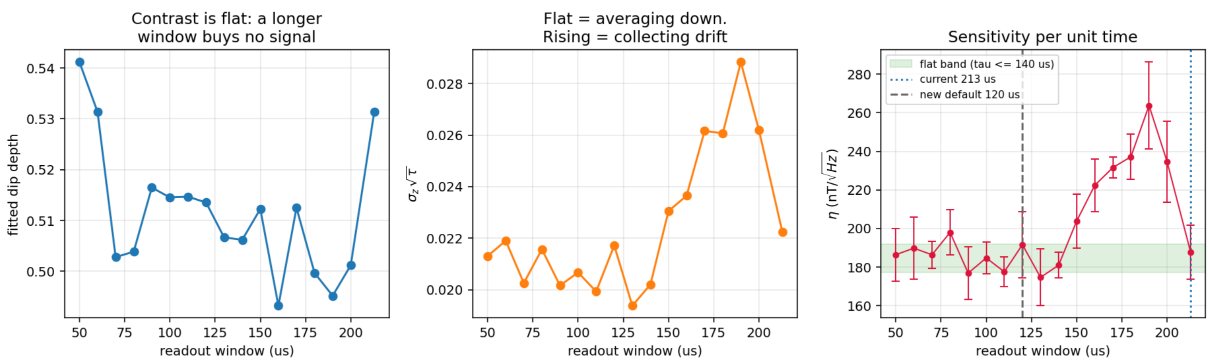

Figure 8.1: Contrast, noise scaling and sensitivity against readout window, 2026-08-06.

Decision: $\tau = 120$ µs. Inside the flat band, and 1.75× faster *per rep* than 213 µs. Note carefully that this does **not** make Method A faster (Chapter 7) — it makes burst and streaming faster, and it lets more reps fit under Method A's call floor.

Two caveats that remain

The 213 µs point is an outlier. It reads 188 nT/$\sqrt{\text{Hz}}$, well below the 150–200 µs trend it should continue, on an unusually low σ_z (1.52e-3). It was the configured default, so those sweeps were probably taken under different conditions. The $\sigma_z\sqrt{\tau}$ trend across 150–200 µs is the more reliable signal.

The shape assumes `reps` was constant across the scan. That is the natural reading of a one-variable scan, and it is what flat $\sigma_z\sqrt{\tau}$ implies, but `reps` is not recorded in the sweep CSVs and cannot be recovered from file timestamps (those gaps are dominated by operator cadence: median 4 s while t_{rep} varies 4×). A reps change part-way through could mimic the rise above 140 µs.

8.2 Measured sensitivity, per method

From `tables/modes.csv`. Both conventions of §2.3 are given; when they disagree, trust the floor.

Session	Method	Run	Rate	$\sigma(\Delta f)$	η	Floor
08-14	averaged	144602	87.8 Hz	56.4 kHz	304	278
08-14	averaged	144655	88.8 Hz	98.1 kHz	526	501
08-14	averaged	145010	87.9 Hz	92.1 kHz	496	486
08-14	averaged	145321	86.8 Hz	54.6 kHz	296	274
08-14	**stream**	145442	**1041.7 Hz**	160.8 kHz	251	240
08-14	**burst**	150458	4166.7 Hz (3089 net)	71.4 kHz	56	**38**
08-06	averaged	144936	86.7 Hz	95.0 kHz	515	499
08-06	burst	145051	2347 Hz (2108 net)	256.1 kHz	267	158

(η and floor in nT/$\sqrt{Hz}$.)

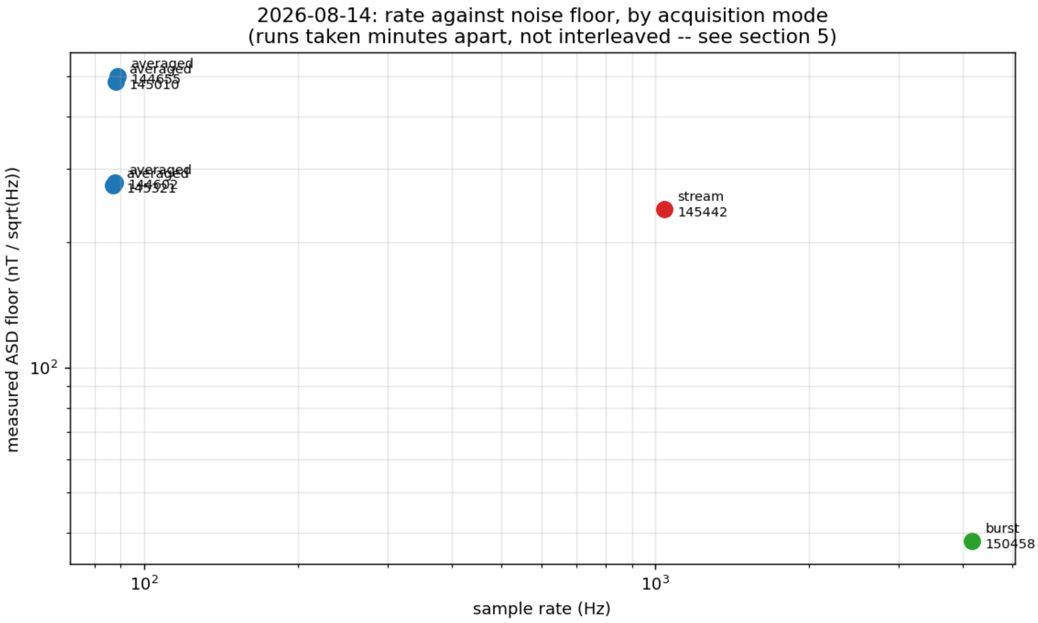

Figure 8.2: Rate against measured noise floor by method, 2026-08-14. The runs were taken minutes apart rather than interleaved — see the warning below.

8.2.1 The photodiode fix: 4.1×

The headline sensitivity result

Burst-mode noise floor, same two parked points, same measurement:

$$\text{2026-08-06: } 158\,\mathrm{nT}/\sqrt{\mathrm{Hz}} \quad\longrightarrow\quad \text{2026-08-14: } \mathbf{38}\,\mathrm{nT}/\sqrt{\mathrm{Hz}} \qquad (\mathbf{4.1}\times)$$

attributable to the reference-photodiode gain correction and the removal of signal-diode saturation. Burst mode is the only method that currently shows the full benefit.

8.2.2 A warning about comparing these rows

These runs are not strictly comparable

They were taken minutes apart, not interleaved, and §8.4 shows the per-rep noise varied **3.6**× between two averaged runs **three minutes apart** with identical settings (145010 vs 145321: 5.97% vs 2.45%).
Any A/B comparison across separate runs on this rig is unreliable. Only interleaved measurements settle a method comparison, which is why the rig test in §11.1 interleaves. Treat the method ranking above as indicative.

8.3 Averaging does not buy $\sqrt{N}$

Block-averaging the real burst samples within each burst, 2026-08-06:

reps averaged	$\sigma(\Delta f)$ **kHz**	**ideal** $1/\sqrt{N}$	**excess**
1	248.6	248.6	1.00
4	141.0	124.3	1.13
8	110.8	87.9	1.26
23	**86.7**	**51.8**	**1.67**
64	76.5	31.1	2.46
128	72.3	22.0	3.29

The predicted 86.7 kHz at $N = 23$ matches the averaged runs, which came in at 73–110 kHz for 11 of 12 runs (median 94 kHz) — an independent consistency check, since the two are measured in completely different modes. But the excess grows with N: past ∼30 reps, averaging is nearly free of benefit.

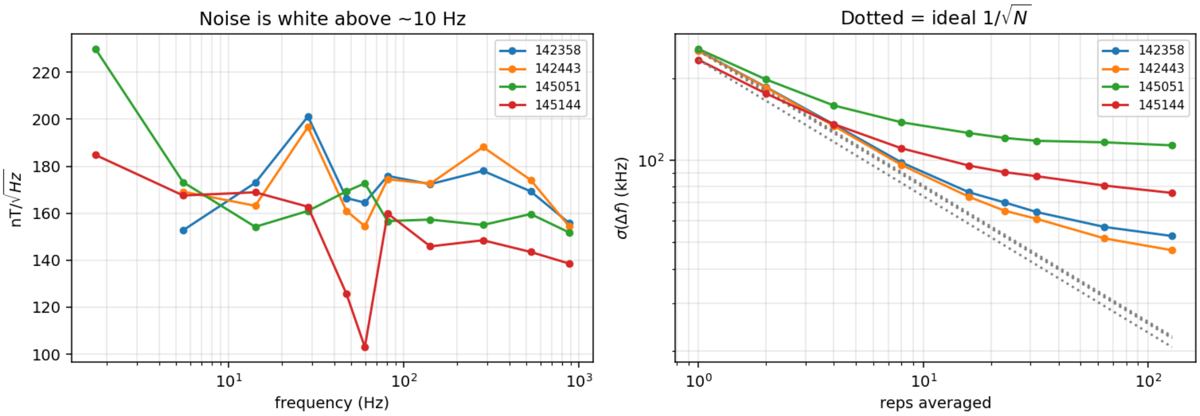

Figure 8.3: Noise spectrum and block-averaging behaviour, 2026-08-06. Above ∼10 Hz the spectrum is white at ∼165 nT/$\sqrt{\text{Hz}}$; the excess is all below 10 Hz.

8.4 The noise is not photon shot noise

This is the sharpest version of the same finding, and the most important open item in the document. Per-rep normalised-PL noise, scaled back from whatever averaging each run used (`tables/per_rep_noise.csv`):

Session	Method	reps averaged	per-rep $\sigma(z)$	relative
2026-08-06	burst	1	0.0076	**0.67%**
2026-08-06	burst	1	0.0091	**0.83%**
2026-08-06	averaged	23	0.0114	1.11%
2026-08-06	averaged	23	0.0161	1.38%
2026-08-14	**burst**	1	**0.0059**	**0.61%**
2026-08-14	averaged	10	0.0214	2.45%
2026-08-14	averaged	10	0.0513	5.97%
2026-08-14	averaged	23	0.0755	**8.95%**
2026-08-14	stream	4	0.0522	7.57%

Two observations:

1. **Burst is reproducibly 0.61–0.83% in both sessions; everything else is 1.1–9%.** Burst measures the per-rep noise *directly*; the other rows infer it by scaling up a reps-averaged σ. The gap means the inference is wrong — the noise in those modes is not a per-rep term.
2. **Averaged mode gets *worse* with more reps** (10 reps → 2.45%, 23 reps → 8.95%). Shot noise cannot do that.

OPEN — the per-acquire noise term

The dominant noise term is **per-acquire, not per-rep**, so it survives averaging entirely. **If it were removed, Method A would inherit Method B's 4× better noise at no cost in rate**, and free averaging (§7.3) would deliver its full $\sqrt{N}$.
Candidate mechanisms, none tested: a start-of-batch transient whose amplitude varies with the (jittery) idle gap between calls; ADC offset drift re-sampled per call; laser/MW state settling after the gap. The row-0 transient measured in burst mode (+1.4%) is direct evidence that *something* happens at a batch boundary.
Protocol: §11.1, check S6.

8.5 Mains pickup and aliasing

From `tables/spectra.csv`:

Method	Rate	Strongest line	Excess over floor	60 Hz appears at
averaged	86.8–88.8 Hz	—	—	**26.8–28.8 Hz (aliased)**
stream	1041.7 Hz	61.7 Hz	**10.2×**	60 Hz
burst	4166.7 Hz	58.5 Hz	3.0×	60 Hz

The mains line is real and strong — **10× the noise floor** in the streamed data, with odd harmonics at 185, 308 and 465 Hz.

At 88 Hz, 60 Hz folds to ∼27 Hz and becomes unfixable

Nyquist at 87.9 Hz is 43.9 Hz, so a 60 Hz tone aliases to $|60 - 87.9| = 27.9$ Hz — the middle of the measurement band. Once folded there it is indistinguishable from a real 27.9 Hz signal, and notching it would remove real signal along with it.

This is a structural argument for running at ≥1 kHz for drone and MCG work, *independent of sensitivity*: only there can the pickup be identified for what it is and removed. `twopoint_spectra.alias_report()` raises this automatically and Step 5A prints it on every averaged run.

Note that the 2026-08-06 analysis reported "no 60 Hz line". That was correct for what it could see: at 87.5 Hz the line is not *at* 60 Hz, and the burst runs of that session, which could have resolved it, were half replays with a badly compressed time axis. The line was there all along.

Chapter 9

Open defects

9.1 Burst mode replays about half of every run

9.1.1 The evidence

Every burst run in both sessions shows the same signature: acquires strictly alternate short and long, and the short ones return a *bit-identical* copy of the previous batch.

Run	Batches	Stale	Fraction	Pattern	Recorded	True
2026-08-06 _142358	211	106	50.2%	13 / 425 ms	3911 Hz	**1946 Hz**
2026-08-06 _142443	66	33	50.0%	13 / 424 ms	4182 Hz	**2091 Hz**
2026-08-06 _145051	44	23	52.3%	13 / 425 ms	4421 Hz	**2110 Hz**
2026-08-06 _145144	14	7	50.0%	13 / 425 ms	4455 Hz	**2227 Hz**
2026-08-14 _150458	**371**	**185**	**49.9%**	12 / 120 ms	6161 Hz	**3089 Hz**

The separation is clean with nothing in between: the duplicate fraction is **0.957 in the fast batches** and 0.069 in the real ones. Within a stale batch the only rows that differ from the previous batch are row 0 and a block around rows 770–818 — and 819 is exactly the streamer's transfer stride, $0.1 \times$ `avg_maxlen/reads_per_shot`.

Detection uses two independent tests, either of which condemns a batch:

1. $\geq$90% of rows identical to the previous batch;
2. wall-clock duration under half the FPGA work the batch claims to contain.

Both are needed. On `_145051`, batch 24 is 82% duplicate — under the fraction threshold, because the run's one CSV-flush stall shifted the alignment — but returned in 12.9 ms against 425 ms of claimed pulse work.

9.1.2 The 2026-08-06 root cause, and why it is now disproved

DISPROVED — `config_all` / `reset_tproc` was not the mechanism

The 2026-08-06 analysis found a real bug. `qickdawg` called `config_all(..., load_pulses=...)`, but `qick` 0.2.386 names that parameter `load_envelopes`. **Every acquire raised `TypeError`**, was silently caught, and fell back to `config_all(soc)` with `reset=False` — and `reset=False` means `stop_tproc(lazy=True)`, which `qick` documents as a no-op on tProc v1. **The tProc was never stopped between runs**, so the accumulated buffer was re-armed under a possibly-still-running program.

That was fixed on 2026-08-12 (commit `942d9e8`) and a `reset_tproc=` argument was threaded through `acquire()`.

The 2026-08-14 run used the fix and the replay fraction did not change: 49.9% (185/371). The rig was definitely running the new code — $\tau = 120\,\mu s$ took effect and the new streaming cell produced output.

So the keyword bug was real and worth fixing, but it was *not* the cause of the replay. **The root cause is unknown.**

What the elimination does tell us: the mechanism is not the tProc failing to stop, because forcing the stop changes nothing. The remaining candidates involve the accumulated buffer or the shot counter rather than the program:

- `config_bufs()` re-arming the accumulator without clearing the counter, so `poll_data` returns the previous run's completed shots as if they were new;
- the board-side worker thread from the previous `start_readout` not being joined, so its queue still holds the old transfer;
- `start_readout`'s shot counter being read before the tProc has reset it.

None has been tested. A single instrumented burst run on the rig, printing the shot counter and the queue depth before and after each `start_readout`, would discriminate.

9.1.3 Mitigation, which is in place

`cfg.multipoint_on_stale = "drop"` makes the freshness guard re-acquire (`multipoint_stale_-retries`, default 2) and, if every attempt still returns the same buffer, return `None` so the caller **skips the batch**. A replay is never written to the CSV as though it were data.

This costs duty cycle, not correctness — the recorded rate becomes honest: 3089 Hz rather than the 6161 Hz the raw file claims. Simulated against a board that replays every second call, all 20 requested batches were recovered as real data.

The guard is a byte comparison (`blake2b`) against the previous raw buffer, so it costs microseconds and catches the failure *whatever* its mechanism — which is why it survived the root cause being wrong.

9.2 Streaming's noise floor is about 6× the burst floor

9.2.1 The evidence

Streaming and burst ran ten minutes apart at the same τ, on the same hardware, with nothing changed on the bench (confirmed with the operator). Their amplitude spectral densities differ by a **flat factor of ∼6 across the whole band** (`tables/read_path.csv`):

Band	Stream	Burst (cleaned)	Ratio
10–30 Hz	234.8	38.6	6.08
30–60 Hz	234.1	44.1	5.31
60–100 Hz	240.8	39.8	6.06
100–200 Hz	231.1	38.7	5.97
200–350 Hz	242.4	38.8	6.25
350–500 Hz	250.2	39.7	6.30

($nT/\sqrt{Hz}$.)

Figure 9.1: Streaming against burst amplitude spectral density. Both floors are flat; the ratio is constant across two and a half decades.

9.2.2 What has been ruled out, with the measurement

Candidate	Measurement	Verdict
Mains pickup	The 60 Hz comb (8 harmonics) accounts for **4.5%** of the stream's Δf variance	ruled out
Slow drift / the PL droop	Below ~10 Hz is **2.8%** of the Δf variance (it is 85% of the *common-mode* variance, but that cancels)	ruled out
Dropped reps	**0** `rep_index` gaps in 125 000 reps	ruled out
Duplicated samples	**0** duplicate rows in 31 250 samples	ruled out
Packet-boundary artefacts	σ at the first 5 rows of a packet is **0.96**× the mid-packet σ (30.7 vs 31.8 ADC)	ruled out
Buffer overflow as packets fill	The 500-sample first packet is *no noisier* than the 205-sample ones	ruled out
Torn reads (partial accumulations)	The residual is symmetric, skew **+0.12**, kurtosis 1.24. Torn reads would skew strongly low	ruled out

A flat, structureless, frequency-independent multiplicative excess is what a **read-path** difference looks like, not what a physical disturbance looks like. Both paths pull from the same accumulated buffer; they differ in how they configure and drain it.

9.2.3 What has been done about it

`stream()`'s configuration sequence now mirrors `NVAveragerProgram.acquire()` step for step. It did not before:

	`acquire()`	`stream()`, before 2026-08-16
`config_all`	yes	yes
`start_src`	yes	yes
`config_bufs`	yes	yes, *different order*
`reload_mem`	**yes**	**missing entirely**
`start_readout`	yes	yes

That is the cheapest candidate to eliminate and it costs nothing if irrelevant. `stream()` also now refuses to reshape a packet whose payload size does not match its own rep count, which would misalign every subsequent sample and mix the two parked frequencies — a failure that would produce exactly this signature and that was previously invisible.

OPEN — neither change is verified to fix it

Highest-priority open item. The test is designed and takes about a minute:

```
python scripts/profile_twopoint_acquire.py -compare-read-paths -tau 120 -reps 500
```

It alternates `acquire(per_rep=True)` and `stream()` on the *same program object*, five times, within seconds. Identical FPGA work; only the read path differs. It compares raw normalised counts rather than kHz, so it does not depend on the ODMR calibration still being valid.

- ratio $\approx 1.0 \Rightarrow$ the $\sim 6\times$ was the environment after all, and streaming needs no caveat;
- ratio $\gg 1 \Rightarrow$ streaming really is noisier on identical work, and it is a software defect. Follow with `-stream-scan` to see whether it grows with buffer fill.

9.3 An interrupted cell wedges the board, and a kernel restart does not clear it

Found 2026-08-17, when Step 4C hung at its auto-zero `acquire()` and `Ctrl-C` produced a traceback ending in `sock.recv(size, socket.MSG_WAITALL)`. Fixed the same day. It is recorded here because it silently contaminates data rather than only costing time, and because the recovery is not obvious.

9.3.1 The mechanism

`qick`'s `poll_data()` takes a `timeout` argument that defaults to `None`, and its board-side body is

```
length, data = streamer.data_queue.get(block=True, timeout=timeout)
```

so with the default it blocks *forever*. `qickdawg`'s drain loop called `qd.soc.poll_data()` with no arguments, so every acquire inherited that default. The queue is fed by the board's readout worker, which only transfers once the tProc shot counter advances past a stride. If the counter never advances — a program that did not start, a tProc left running by an earlier cell, an abandoned `stream()` generator — nothing is ever queued and the RPC never replies.

The client then sits in `sock.recv(..., MSG_WAITALL)` waiting for a reply that is not coming. That is the traceback above, and the four consequences are:

1. **The wait is unbounded.** No timeout, anywhere in the path.
2. **`Ctrl-C` only unblocks the client.** The board-side Pyro worker thread stays blocked in `queue.get()` for the life of the daemon.
3. **That thread is still a consumer.** When the next run finally queues a packet, the abandoned `get()` can take it instead of the live caller — so a subsequent run silently loses its first packet.
4. **Restarting the notebook kernel does not help**, because the wedged thread is on the board, not in the client.

9.3.2 Reproduced offline

Against the real `qick.streamer.DataStreamer` driven by a fake board whose shot counter never advances (`scripts/` is not needed; the check is eight lines):

call	argument	result
`poll_data()`	`timeout=None` (the default)	never returned; killed at 3 s
`poll_data(totaltime=0.1, timeout=1.0)`	finite	returned `[]` after 1.01 s

9.3.3 The fix

title=Three changes, all shipped

1. **Every poll is bounded.** `NVAveragerProgram._drain_readout()` (extracted from `acquire()`, so it is now testable on its own) and `stream()` both pass an explicit `timeout`. A stalled readout raises `TimeoutError` naming the shot count instead of hanging. This is also what makes `stream()`'s `poll_timeout_s` parameter work — it was unreachable before, since the call it guards never returned.
2. **Interrupts clean up.** The drain loop catches `BaseException` (so, `Ctrl-C` too) and `stream()` has a `finally`, which for a generator also runs on `GeneratorExit` — the `break`, the interrupt, and garbage collection of a partly-consumed stream. Both call `_abort_readout()`: stop the tProc, then stop the readout.
3. **An explicit recovery.** `MultipointLockinODMR.recover()` stops the tProc, stops the readout, and calls `DataStreamer.start_worker()`, which installs fresh queues and flags so anything blocked on the old queue is orphaned against an object nothing will ever write to again. `ensure_idle()` is the one-RPC pre-flight now at the top of Steps 4A, 4B and 4C; it escalates to `recover()` only when the board is actually dirty, because a full recover per run leaks a board-side worker thread.

title=Still open: does this explain the burst replay?

Consequence 3 above is a mechanism by which one run's packet is delivered to a *different* run's consumer. That is the first concrete candidate for §9.1 since `reset_tproc=True` was eliminated, and it fits the alternating short/long signature: a call that receives a packet meant for the previous run returns early with stale contents.
It is **not** established. The 2026-08-14 burst run was not obviously preceded by an interrupt, and a wedge should not recur once per two batches for 371 batches. Testable directly: run a burst with the fix in place and re-measure the stale fraction. If it drops from ∼50% to zero, the two defects were one defect.

9.4 Averaged and burst runs used to overwrite each other's filenames

Step 4A and Step 4B both wrote `twopoint_lockin_live_<stamp>.csv`. Nothing was lost — the timestamps differ — but an averaged run and a burst run were indistinguishable by name, which is why identifying which recorded file was which mode has repeatedly needed column inspection rather than a glance at the directory.

Fixed 2026-08-17: Step 4A writes `twopoint_lockin_avg_*`, Step 4B writes `twopoint_lockin_burst_*`, Step 4C already wrote `twopoint_lockin_stream_*`. `find_live_csvs()` in `analyze_twopoint_session.py` accepts all three prefixes so previously recorded sessions still load unchanged. Files already on disk keep their `_live_` names and are still found; only the mode has to be read from the columns, which `detect_mode()` does anyway.

9.5 Post-processing: recovering runs already recorded

Independently of any hardware fix, existing burst files can be made usable. The pipeline is `process_burst()` in `twopoint_postprocess.py`, reachable from the command line as `scripts/clean_burst_lockin.py`. It applies, in order:

1. **Stale-batch removal**, by the two tests above. Reports the fraction.
2. **Row-0 transient removal** from every batch.
3. **Re-timing on the exact cadence** from §3.3, with each surviving burst marked as its own `segment` so the inter-burst dead time is an explicit gap. Nothing is interpolated across it and spectra are computed per segment.
4. **Differential despiking** — a causal Hampel filter on `peak_shift_kHz`.

Order matters. Staleness is judged on the *unfiltered* frame, because the batch table reconstructs each batch's start time from its first row; dropping row 0 first would shift every start by half a sample.

What it cannot do. The samples lost to the dead time between bursts are not there, and the genuine single-rep glitches that were double-counted are *de*-duplicated, not explained. It recovers a correct 3.1 kHz record from a 6.2 kHz one; it does not recover 6.2 kHz.

9.5.1 A bug this found: never despike the channels separately

Per-channel despiking makes the data worse

The two parked channels are **0.84–0.97 correlated** through the common-mode PL. A Hampel filter run on them independently flagged **30 and 23** samples with only **4 in common** — so most replacements are one-sided, and each one-sided replacement breaks the cancellation the estimator depends on and *injects* differential noise.
Measured effect:

	raw	**per-channel**	**differential**
averaged 145321	54.6 kHz	55.8 kHz	**53.6 kHz**
stream 145442	160.8 kHz	175.5 kHz	**155.2 kHz**

The default is now `despike="shift"`, which filters the differential. Per-channel remains available as `despike="channels"` for diagnosing raw-channel glitches, and prints a warning saying it will usually raise σ.

The live-loop despiker is a separate case and legitimately per-channel: there, the raw channels must be cleaned *before* the conversion exists. It is off by default (`AVG_DESPIKE_ENABLED = False`).

Chapter 10

Work log: what has been changed and pushed

Repository: `github.com/chalach49266/nv-compact-magnetometer`, branch `main`. All commits below are pushed. The two sessions are separated because the second one partly retracts the first.

10.1 Round 1 — 2026-08-12, against the 2026-08-06 session

Commit	Scope	What it did
`5ea75ab`	`data:`	2026-08-06 session snapshot (drone sweeps, burst runs, 17-point integration-time scan).
`50fe514`	`docs:`	The timing & noise analysis, plus `analyze_twopoint_session.py` and `profile_twopoint_acquire.py`.
`942d9e8`	`perf:`	Corrected the `config_all` keyword; τ 213 → 120 µs; added ABBA ordering; append-only CSV writer.
`6765284`	`fix:`	`MultipointLockinODMR.stream()` — the continuous streaming path.
`9eed2e1`	`feat:`	`clean_burst_lockin.py`, to salvage burst runs already recorded.
`16fd9ba`	`docs:`	Changelog, rig checklist, cross-references.
`f700223`	`fix:`	Sensitivity omitted the $\sqrt{\text{reps}}$ factor, understating η by $\sim$10×.

Scorecard on round 1. Of the four changes intended to improve things:

Change	Outcome	Verdict
τ 213 → 120 µs	PARTIAL	Took effect and is right for sensitivity, but bought **no rate** — the premise that τ set the rate was wrong.
`config_all` fix + `reset_tproc`	OPEN	Real bug, correctly fixed, but **did not stop the replay**.
`stream()`	DONE	Works: **1041.7 Hz**, gapless. Its noise is a new open defect.
Append-only CSV writer	DONE	Removed the 25–220 ms periodic stalls.
ABBA ordering	OPEN	Implemented, never tested on hardware.

10.2 Round 2 — 2026-08-16/17, against the 2026-08-14 session

Commit	Scope	What it did
`a98fd2f`	`data:`	2026-08-14 session (101 MB): four averaged runs, one stream run, one burst run all at τ=120 µs; a 12-point integration-time sweep (1–213 µs, 5 repeats each); an 8-peak reference sweep; a multipoint run. Committed *before* any code change, so the analysis has a fixed baseline.
`6eb7eb5`	`docs:`	**Retraction** of the “rate is FPGA-bound” conclusion in the 2026-08-06 document, with the evidence, plus marking §4.3’s root cause disproved.
`524dbb4`	`feat:`	`twopoint_spectra.py` (365 lines) and `twopoint_postprocess.py` (790 lines); `clean_burst_lockin.py` rewritten to delegate. Found and fixed the per-channel despiking bug and the 12% burst-cadence error.
`6893bcb`	`perf:`	Corrected rate model (`HOST_CALL_S`/`HOST_FLOOR_S` replacing `HOST_OVERHEAD_S`); `reps_that_fit()`; `measure_host_floor()`; `multipoint_on_stale="drop"`; `stream()` sequence aligned with `acquire()` and packet-size validation.
`8fd303f`	`refactor:`	Notebook split into Step 4A/5A, 4B/5B, 4C/5C, Step 6. New `twopoint_runner.py` (236 lines) for the shared plumbing. 1497 lines of notebook changed.
`68e95a9`	`feat:`	Three rig tests: `-compare-read-paths`, `-stream-scan`, `-floor`.
`f1bd0eb`	`docs:`	The 2026-08-14 methods document, `analyze_twopoint_0814.py`, five generated tables and three figures; pointer fixes in `LOCKIN_ACQUIRE.md` and `twopoint_lockin_notes.md`.

10.3 Round 3 — 2026-08-17, the board wedge

Triggered by a live failure at the rig rather than by analysis: Step 4C hung at its auto-zero `acquire()`, and `Ctrl-C` would not release it. Diagnosed to `poll_data()`’s `timeout=None` default and fixed the same day. Full account in §9.3; the naming collision in §9.4 was found while tracing which file each mode had written.

Scope	What it did
`fix(qickdawg):`	`acquire()`’s drain loop extracted to `_drain_readout()` and bounded: an explicit `poll_data` timeout, a no-progress deadline that raises `TimeoutError`, and a `BaseException` handler that stops the tProc and the readout before re-raising. Adds `POLL_TOTALTIME_S`, `POLL_TIMEOUT_S`, `ACQUIRE_IDLE_S`, `_acquire_timeout_s()` and `_abort_readout()`.
`fix(twopoint):`	`stream()` passes the same explicit timeout — which is what finally makes its own `poll_timeout_s` reachable — and wraps its loop in `try/finally`, so an interrupted or abandoned generator leaves the board idle. Records `stream_qc()["aborted"]`.
`feat(twopoint):`	`MultipointLockinODMR.recover()` and `ensure_idle()`. The pre-flight is wired into Steps 4A, 4B and 4C.
`fix(notebook):`	Step 4A writes `twopoint_lockin_avg_*` and Step 4B `twopoint_lockin_burst_*` instead of both writing `_live_`; `find_live_csvs()` accepts all three prefixes.

10.3.1 Verified offline

No board was available, so all three paths were exercised against fakes built from the real `qick` sources:

Check	What it drives	Result
Root cause	real `DataStreamer`, counter frozen; `poll_data()` vs `poll_data(timeout=1.0)`	hangs vs returns in 1.01 s
Healthy drain	`_drain_readout()` from the edited file, 50 shots in 10-shot packets	50/50, buffer bit-exact
Stalled tProc	same, board returns nothing	`TimeoutError` at the budget, board cleaned
Interrupt	`KeyboardInterrupt` mid-drain	re-raised, board cleaned
Stream abort	real `stream()` source, consumer breaks after 10 reps	`finally` ran, `aborted=True`
Stream completion	same, run to 20/20 reps	cleaned, `aborted=False`

10.4 The notebook layout

`Modules/Twopoint_Lockin_module.ipynb`, 32 cells. Step 4 used to be a single 26 kB cell with a `LIVE_BURST_MODE` switch, and Step 5 a single generic analysis cell that treated all three methods alike — which is wrong in a different way for each of them.

Cells	Section	Contents
0–8	setup, ODMR sweep	project root, `default_config` (τ set here), DSA attenuation, the reference ODMR sweep
9–12	Step 1a / 1b	pick the peak to track; place the parked pair and build `TWOPOINT_CALIB` (this is where the slopes and baselines come from)
13–16	Step 2 / 3	one batch at the two parked frequencies; apply the conversion
17–18	**Step 4A**	averaged acquisition. Measures the host floor, sizes `reps` to fill it, prints the alias warning
19–20	Step 5A	`process_averaged` + ASD, ~15 lines
21–22	**Step 4B**	burst acquisition, `on_stale="drop"`, cadence-based timestamps, reports `BURST_STALE_FRACTION`
23–24	Step 5B	`process_burst` + per-segment ASD
25–26	**Step 4C**	streaming acquisition, `stream_qc()` assertions
27–28	Step 5C	`process_stream` + ASD, optional mains notch
29–30	Step 6	post-process *any* saved run; mode detected from the columns; side-by-side comparison table
31	Caveats	what the method does and does not measure

Each Step 4x is self-contained — it builds its own program, primes, auto-zeroes and writes its own CSV — with the parameters that change run to run inline, where they are edited. Each Step 5x reads the run its partner just wrote. The shared mechanical plumbing (append-only CSV writer, row building, the timing/freshness/calibration reports, the standard figure) lives in `notebook_modules/twopoint_runner.py` so that 4A and 4B do not carry 150 duplicated lines each.

10.5 The code, file by file

File	State	Role
Acquisition		
`notebook_modules/multipoint_lockin_program.py`	modified	The FPGA program, all three read paths, the timing model, the freshness guard.
`qickdawg/nvpulsing/nvaverageprogram.py`	modified	Vendored; the `config_all` fix and `reset_tproc` plumbing.
`Modules/Twopoint_Lockin_module.ipynb`	restructured	32 cells: setup, ODMR, Steps 1–3, then 4A/5A, 4B/5B, 4C/5C, 6.
Analysis (new)		
`notebook_modules/twopoint_spectra.py`	**new**	Every FFT. One-sided PSD, $\eta = \sigma\sqrt{2\Delta t}$, mains-line finder, alias report, comb notch. Verified against Parseval.
`notebook_modules/twopoint_postprocess.py`	**new**	One pipeline per mode + mode detection + exact readout-quantum recovery + CLI.
`notebook_modules/twopoint_runner.py`	**new**	Shared acquisition plumbing: append-only CSV, row building, the three reports, the standard figure.
`notebook_modules/burst_qc.py`	reused	Batch table, stale masks, retiming, segment PSD, block averaging.
`notebook_modules/spike_rejection.py`	reused	`HampelDespiker`.
Scripts		
`scripts/analyze_twopoint_0814.py`	**new**	Regenerates every number and figure in this document.
`scripts/profile_twopoint_acquire.py`	modified	Static signature check + per-RPC timing + the three rig tests.
`scripts/clean_burst_lockin.py`	rewritten	Burst CLI, now delegating to `twopoint_postprocess`.
`scripts/analyze_twopoint_session.py`	unchanged	The 2026-08-06 analysis; still regenerates that document.
Documents		
`docs/twopoint_master_reference/`	**new**	This document.
`docs/2026-08-14_twopoint_methods/`	new	Markdown source + generated tables/figures.
`docs/2026-08-06_twopoint_timing/`	annotated	The earlier analysis, with corrections marked in place.
`docs/LOCKIN_ACQUIRE.md`	pointer fixed	Repeated the retracted 1.6 ms figure.
`docs/twopoint_lockin_notes.md`	pointer fixed	Same.

10.6 Wiki

The synthesis is filed in the Obsidian vault at `NV Project/`.

- **New page:** `wiki/sources/twopoint_three_modes_2026_08_14.md`
- **Correction banner:** `wiki/sources/twopoint_timing_and_burst_defect.md`
- **Updated:** `wiki/experiments/lockin_tracking.md` (new "August 14–17" section, and the August 6–12 one partly retracted); `wiki/concepts/sensitivity.md` (three new rows, plus a warning that the two-point noise does not average down); `wiki/questions/open_questions.md`

(three questions answered, four new ones with pass criteria); `wiki/overview.md` (status table and the "Where We Stand" list); `index.md` (routing).

- **Log entry:** `log.md`, dated 2026-08-17.

Not committed to git: the vault sits inside the home-directory repository, which contains unrelated and sensitive untracked files. The files are on disk and sync through OneDrive.

Chapter 11

What still needs to be done

11.1 Rig checks, in priority order

Everything here needs the RFSoC at 192.168.0.103, reachable only from the Windows rig machine. Each has a pass criterion.

#	Check	Command	Pass criterion
S0	**Does the wedge fix close the burst replay?** Now the cheapest decisive test, because §9.3 gives §9.1 its first mechanism since `reset_tproc` was eliminated. *Do this first.* ∼1 min.	Run Step 4B for 30 s and read `BURST_STALE_FRACTION`.	**0%** ⇒ the replay was the wedge and both defects close. Still ∼**50%** ⇒ they are independent; S1 next.
S1	**Burst vs stream, interleaved.** Decides whether the ∼6× streaming excess is the read path or the room. *Do this first.* ∼1 min.	`profile_twopoint_-acquire.py -compare-read-paths -tau 120 -reps 500`	ASD ratio within **1.25**× ⇒ environment. Well above ⇒ software defect.
S2	Does the streaming excess grow with run length (buffer fill)?	`profile_twopoint_-acquire.py -stream-scan`	Floor at 125 k reps within **1.3**× of the floor at 2 k reps.
S3	Confirm the per-call floor and the flat rate.	`profile_twopoint_-acquire.py -floor -tau 120 -reps 1 4 10 23 42 100`	Fastest call **9–11.5 ms** with a `reps=1` probe; period flat to **15%** across reps whose FPGA work fits under it.
S4	**Fill in §3.4** — the per-RPC split, the one structural gap in this document.	`profile_twopoint_-acquire.py -ip 192.168.0.103`	A time for each of stages 1–7. No pass/fail; this is a measurement that has never been made.
S5	Free averaging. 30 s Step 4A with `AVG_REPS_PER_BATCH = None`.	Notebook Step 4A	Still ∼88 Hz *and* $\sigma(\Delta f) \leq$ **54.6 kHz**. Given §8.4, expect less than the naive 2×.
S6	**The per-acquire noise term.** Interleave averaged runs at reps = 10 / 23 / 42 within one sitting.	Notebook Step 4A, three configs, alternating	Does σ fall as $1/\sqrt{\text{reps}}$? If not, the term is per-call — then test whether it tracks the idle gap by varying `SAVE_EVERY_SEC` or inserting a deliberate sleep.
S7	ABBA A/B at fixed τ and equal averaging (`"forward"` 46 reps vs `"abba"` 23 reps), interleaved.	Notebook Step 4A, two configs	Report the change in $\sigma(\Delta f)$. Any reduction is free.
S8	Honest burst rate with the drop policy. 10 s Step 4B.	Notebook Step 4B, `BURST_-ON_STALE="drop"`	Step 5B reports **`n_stale_-batches` = 0** on the file Step 4B just wrote, and `BURST_-STALE_FRACTION` is printed.

#	Check	Command	Pass criterion
S9	Instrument the replay. Print the shot counter and queue depth before/after each `start_readout` during a burst run.	needs a small patch to `acquire()`	Discriminates the three candidate mechanisms in §9.1.

11.2 Software backlog

Item	Priority	Why
Periodic re-zero during a streaming run	high	The 25% PL droop (§6.5) walks the operating point off the calibrated flank over 30 s. Currently only detrended in post.
Root-cause the burst replay	high	Would recover ~15% of burst rate and remove the need for the drop policy.
Real-time mains notch in the acquisition path	medium	The current notch is offline, non-causal, and assumes stationarity. At ≥1 kHz a causal notch is feasible and the line is 10× the floor.
Fold the multipoint notebook onto the same modules	medium	`twopoint_spectra.py` and `twopoint_postprocess.py` are mode-generic; the multipoint path still has its own analysis code.
Calibrated field reference	medium	Every η here is a readout-chain figure (§2.4). A known applied field would turn them into system claims.
`remove_offset` is documented but not implemented	low	`NVAveragerProgram.acquire()` accepts the argument and never uses it. Harmless today (both paths are consistent) but a latent trap.
Allan deviation on the long runs	low	Needed for the drift/averaging-time story; the June time-integration data exists but has not been analysed drift-aware.

11.3 Recommended operating point today

Until S1 resolves the streaming question

For best sensitivity — Method B, burst.
τ = 120 µs, `BURST_REPS = 500`, `BURST_ON_STALE = "drop"`. 38 nT/$\sqrt{\text{Hz}}$ at an effective 3.1 kHz. Accept the ~15 ms gap every 135 ms.

For a continuous trace — Method C, streaming.
τ = 120 µs, `STREAM_TARGET_HZ = 1000`. 1041.7 Hz, gapless, exact timestamps, but ~6× the noise until §9.2 is closed.

For slow work and sanity checks — Method A, averaged.
τ = 120 µs, `AVG_REPS_PER_BATCH = None` so the call floor is filled. ~88 Hz, no open defects — but remember 60 Hz is aliased into the band here.

Chapter 12

Corrections register

Every claim from an earlier document that this one contradicts, in one place.

Source	Earlier claim	Current position	Evidence
2026-08-06 doc §1, §2	"The rate is FPGA-bound, not host-bound"; host round-trip ∼1.6 ms	**Retracted.** Host-bound, ∼10–11.4 ms per call; the FPGA runs underneath it	§7.1
2026-08-06 doc §2.6	τ is "the main lever"; `reps` trades against rate one for one	**Both backwards.** Neither affects the rate below the floor; `reps` is a free *sensitivity* lever	§7.4
2026-08-06 doc §2.4	"Host round-trip 1.60 ms (14.1%)" as a budget row	That was a *residual* under an assumed serial model, not a measurement	§7.1
2026-08-06 doc §4.3	`config_all` / never-stopped tProc is the root cause of the burst replay	**Disproved.** The fix was applied and 49.9% still replayed	§9.1
2026-08-06 doc §3.2	"No 60 Hz line, no discrete tones"	The line is there and is 10× the floor; at 87.5 Hz it is *aliased* to ∼28 Hz and the burst runs that could have resolved it were half replays	§8.5
2026-08-06 doc, 1st draft	$\eta \approx 19\,\mathrm{nT}/\sqrt{\mathrm{Hz}}$ per rep	$185\,\mathrm{nT}/\sqrt{\mathrm{Hz}}$. The draft paired a reps-averaged σ with a single-rep time, omitting $\sqrt{\mathrm{reps}} \approx 10$	§2.3
2026-08-06 doc, η convention	$\eta = \sigma_B\sqrt{\Delta t}$	$\eta = \sigma_B\sqrt{2\Delta t}$, matching what the notebook prints; a $\sqrt{2}$ difference	§2.3
2026-08-14 doc, 1st draft	Streaming excess is "4.2×"	5.3–6.3×. The 4.2 used the *raw* burst floor; the cleaned floor is 38.6 rather than $55\,\mathrm{nT}/\sqrt{\mathrm{Hz}}$	§9.2
Notebook Step 4 comment, pre-2026-08-12	"∼10 ms of fixed host overhead, FPGA work that fits inside it is FREE"	**Substantially correct**, and wrongly "corrected" on 2026-08-12. Restored	§7.2
`time_per_rep()`, pre-2026-08-12	$n_{\mathrm{slots}} \times (\tau + 2\,t_{\mathrm{relax}})$	$n_{\mathrm{slots}} \times \tau$; the relax windows are absorbed	§3.1
Burst cadence, pre-2026-08-16	From `acq_seconds/n_-samples`: 269 µs	240.0 µs exactly, from the readout quantum. The old value included the host share	§3.3
Despiking, pre-2026-08-16	Filter the two parked channels independently	Filter the *differential.* Per-channel raises σ	§9.5.1

The methodological lesson

Four of the twelve rows above come from the same mistake: **fitting a model with n free parameters to data taken at one operating point**. The 2026-08-06 session ran at a single (τ, reps), and every timing conclusion drawn from it was under-determined — the serial-versus-overlapped question simply cannot be answered from one point, however carefully the residuals are computed.

The 2026-08-14 session settled it in an afternoon purely by varying two knobs. Any future timing claim should span at least three operating points before it is written down.

Chapter 13

Reproducing everything

13.1 Offline, no hardware

```
# every number and figure in this document
python scripts/analyze_twopoint_0814.py

# post-process any run; the mode is detected from the columns
python -m twopoint_postprocess <csv> [--outdir DIR] [--notch-mains]

# burst-specific CLI with the original flags
python scripts/clean_burst_lockin.py <live_csv> --outdir analysis/

# the 2026-08-06 analysis, unchanged
python scripts/analyze_twopoint_session.py

# static check of the config_all signature
python scripts/profile_twopoint_acquire.py --check-only

# rebuild this document
cd docs/twopoint_master_reference && latexmk -pdf TWOPOINT_MASTER_REFERENCE.tex
```

13.2 On the rig

```
python scripts/profile_twopoint_acquire.py --floor --tau 120
python scripts/profile_twopoint_acquire.py --compare-read-paths --tau 120 --reps
500
python scripts/profile_twopoint_acquire.py --stream-scan
python scripts/profile_twopoint_acquire.py --ip 192.168.0.103
    # the full per-RPC profile (fills section 3.4)
```

13.3 Data used by this document

Path	**Contents**
`data/results/ 081426 (Sensitivity increase update)/`	2026-08-14: 4 averaged + 1 stream + 1 burst two-point run, all at τ=120 µs; 12-point integration sweep; 8-peak reference sweep; multipoint run. 101 MB.
`data/twopoint_lockin/08062026/`	2026-08-06: drone sweeps, 4 burst runs, 12 averaged runs, 17-point integration sweep (92 ODMR sweeps).
`docs/2026-08-14_twopoint_methods/ tables/`	Generated: `timing.csv`, `mode_budgets.csv`, `modes.csv`, `read_path.csv`, `per_rep_noise.csv`, `spectra.csv`.